

numerlA

Pipeline ML Automatisé pour Numerai

Document de revision technique | Support d'entretien

Architecture | Machine Learning | Cloud AWS | Automatisation

Version 1.0 - Mars 2026

Projet personnel | Hedge Fund Decentralise Numerai

SOMMAIRE

1. Vue d'ensemble du projet
2. Architecture 3 tiers
3. Pipeline d'entrainement (train.py)
4. Modeles ML utilises
5. Post-processing
6. Pipeline de prediction (predict.py)
7. Optimisation des hyperparametres
8. Tournois Numerai
9. Noeuds de prediction cloud

10. Automatisation

11. Diagnostics et evaluation

12. Structure du projet

13. Questions d'entretien et reponses

1. VUE D'ENSEMBLE

numerIA est un pipeline de machine learning end-to-end automatise concu pour participer au tournoi Numerai, un hedge fund decentralise. Le projet couvre l'ensemble de la chaine de valeur : ingenierie des features, entraînement de modeles ensemblistes, prediction, deploiement cloud (AWS) et soumission automatique aux 3 tournois Numerai (Classic, Signals, Crypto) via 4 noeuds de prediction.

POINTS CLES DU PROJET

- Objectif** : Predire les rendements d'actifs financiers de maniere automatisee
- Methode** : Ensemble multi-modeles (LightGBM + XGBoost) avec stacking Ridge
- Deploiement** : AWS (EC2 Spot, Lambda, ECR, S3, EventBridge)
- Cout** : ~0.40 a 0.80 EUR par execution hebdomadaire
- Tournois** : Classic (regression), Signals (actions), Crypto (crypto-actifs)

Composant	Technologie	Role
Entraînement	LightGBM, XGBoost, GPU CUDA	Modeles ensemblistes multi-cibles
Feature Engineering	Python, NumPy, Pandas	37 features synthetiques creees
Validation	CV temporelle era-based	5 folds + embargo 4 eras
Stacking	Ridge Regression	Agregation optimale des modeles
Deploiement	Docker + numerai-cli	4 noeuds de prediction cloud
Infrastructure	AWS Terraform	EC2 Spot, Lambda, ECR, S3
Automatisation	PowerShell, EventBridge	Pipeline quotidien/hebdomadaire

2. ARCHITECTURE 3 TIERS

L'architecture du projet est organisee en 3 tiers distincts, separant le developpement local, la conteneurisation et le deploiement cloud.

2.1 Tier 1 - Developpement local (Windows 11 + GPU RTX 4060)

ENVIRONNEMENT LOCAL

OS : Windows 11 avec GPU NVIDIA RTX 4060
CUDA : Version 12.6 pour l'acceleration GPU
Environnement : Conda 'lgbm-gpu' (Python 3.10/3.11)
Fonctions : Entrainement des modeles, feature engineering, preprocessing
Automatisation : Scripts PowerShell (run_me.ps1, daily_*.ps1)

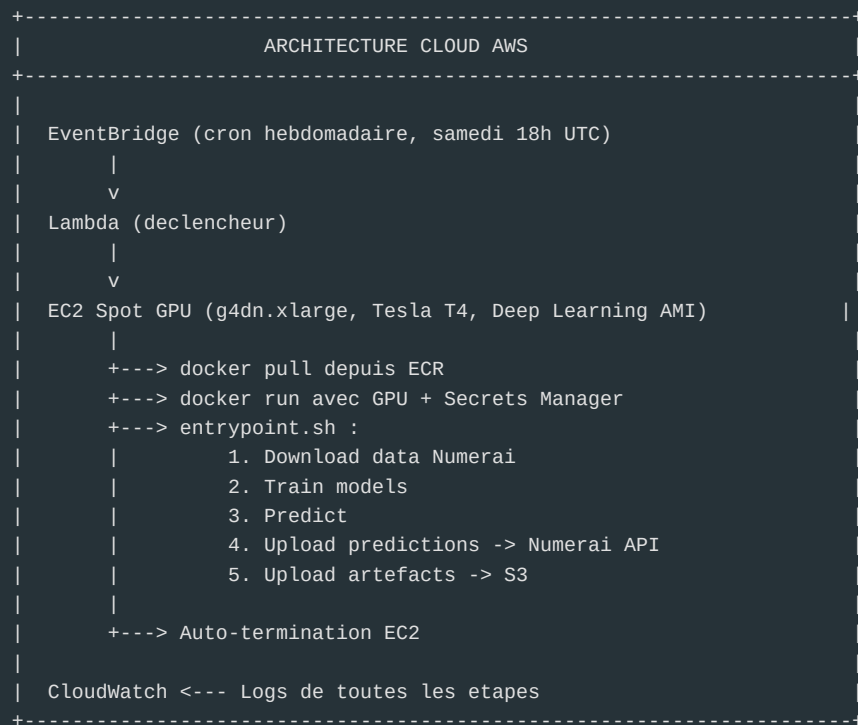
2.2 Tier 2 - Conteneurisation (Docker + numerai-cli)

Chaque noeud de prediction est encapsule dans un conteneur Docker independant contenant : un Dockerfile base sur Python 3.11, un script predict.py, le modele pre-entraine (.pkl) et les dependances. 4 noeuds sont deployes : Classic, Signals, Crypto et TGRV2.

2.3 Tier 3 - Cloud AWS (Terraform)

Service AWS	Role	Detail
EC2 Spot GPU	Entrainement cloud	g4dn.xlarge, Tesla T4
Lambda	Declencheur	Demarre l'instance EC2
EventBridge	Scheduling	Cron hebdomadaire (sam. 18h UTC)
ECR	Registre Docker	Stockage des images Docker
Secrets Manager	Securite	Cles API Numerai
S3	Stockage	Artefacts (retention 90 jours)
CloudWatch	Monitoring	Logs et metriques

Diagramme d'architecture AWS



COUT ESTIME

L'utilisation d'instances EC2 Spot permet de reduire les couts de ~70% par rapport aux instances on-demand. Cout moyen par execution hebdomadaire : **0.40 a 0.80 EUR**. Les services Lambda et EventBridge sont pratiquement gratuits a cette echelle.

3. PIPELINE D'ENTRAINEMENT (train.py)

Flow du pipeline d'entrainement

```
+-----+
|                PIPELINE D'ENTRAINEMENT                |
+-----+
|
| [1] Chargement configs YAML                             |
|   | training.yaml, program_input_params.yaml, features.yaml |
|   v                                                         |
| [2] Chargement donnees Numerai                         |
|   | numerai_training_data.parquet (~300k lignes, ~2000 feat) |
|   v                                                         |
| [3] Selection des features                             |
|   | Filtrage NA (max 10%), correlation min, cap max         |
|   v                                                         |
| [4] Feature Engineering (+37 features)                 |
|   | Row stats (11) + Group stats (16) + Interactions (10)   |
|   v                                                         |
| [5] Cross-Validation temporelle                        |
|   | 5 folds, embargo 4 eras, holdout 50 eras recentes       |
|   v                                                         |
| [6] Entrainement ensemble par cible                    |
|   | 5x LightGBM + 1x XGBoost [+ Ridge, MLP opt.]           |
|   v                                                         |
| [7] Stacking (Ridge sur predictions OOF)              |
|   |                                                         |
|   v                                                         |
| [8] Sauvegarde modeles .pkl + metadata JSON            |
+-----+
```

3.1 Feature Engineering

Le module de feature engineering cree 37 features synthetiques a partir des features originales de Numerai. Ces features capturent des patterns statistiques complementaires.

Type de Feature	Nombre	Description
Row Statistics	11	mean, std, min, max, range + 5 bin counts par ligne
Group Statistics	16	8 groupes de features, mean et std par groupe
Interactions	10	10 paires de differences entre features selectionnees
TOTAL	37	Features synthetiques ajoutees aux features originales

3.2 Cross-Validation temporelle

STRATEGIE DE VALIDATION

Methode : Time-series split era-based (5 folds)

Embargo : 4 eras entre train et validation pour eviter le data leakage temporel

Holdout : 50 eras les plus recentes reservees pour l'evaluation finale

Justification : Les donnees financieres sont temporellement auto-correlees. Une validation classique (random split) surestimerait les performances.

Schema de la cross-validation temporelle

Era 1 Era 50 Era 100 Era 150 Era 200 Era 250 Era 300
|-----|-----|-----|-----|-----|-----|

Fold 1: [TRAIN=====]...[VAL]
Fold 2: [TRAIN=====]...[VAL]
Fold 3: [TRAIN=====]...[VAL]
Fold 4: [TRAIN=====]...[VAL]
Fold 5: [TRAIN=====]...[VAL]

Holdout: [=====TRAIN=====] [HOLDOUT=====]

... = embargo de 4 eras (gap pour eviter le data leakage)

4. MODELES ML UTILISES

4.1 Modeles de base

4.1.1 LightGBM (modele principal)

LightGBM est le modele principal du pipeline. Il utilise une strategie de croissance leaf-wise (par feuille) qui est plus efficace que la croissance level-wise pour les grands datasets. 5 instances avec des seeds differentes (42-46) sont entraînées pour la diversite.

Hyperparametre	Valeur	Explication
num_leaves	512	Complexite de l'arbre (leaf-wise)
min_data_in_leaf	3000	Regularisation contre l'overfitting
lambda_l2	10.0	Regularisation L2
feature_fraction	0.1	10% des features par arbre (diversite)
bagging_fraction	0.7	70% des donnees par iteration
learning_rate	0.005	Taux d'apprentissage faible (precision)
n_estimators	10000	Max iterations (early stopping a 200)
device	gpu	Acceleration GPU CUDA

4.1.2 XGBoost (diversite)

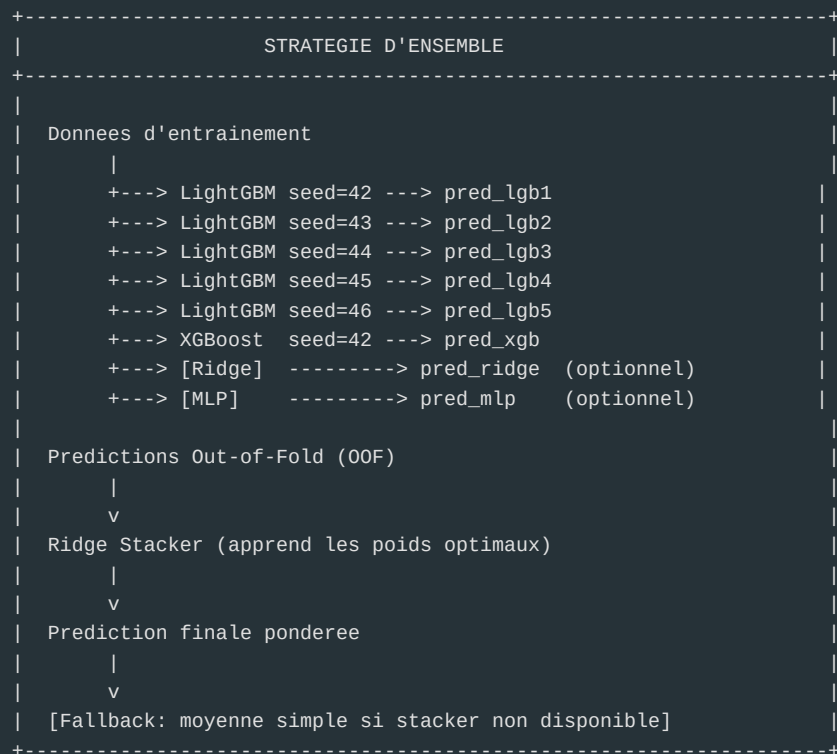
XGBoost apporte de la diversite a l'ensemble grace a sa strategie de croissance level-wise (par niveau). Les parametres sont mappes depuis la configuration LightGBM pour assurer la coherence. Utilise gpu_hist comme methode d'arbre.

4.1.3 Modeles optionnels

Ridge Regression : baseline lineaire servant de reference. **MLP (Multi-Layer Perceptron)** : reseau de neurones via scikit-learn MLPRegressor, apportant une diversite supplementaire par son approche non-arborescente.

4.2 Strategie d'agregation (Stacking)

Architecture de l'ensemble avec stacking



4.3 Strategie Multi-Target

GESTION MULTI-CIBLES

Detection automatique de toutes les colonnes target_*

Cap : 20 cibles maximum

Ensemble separe par cible (chaque cible a son propre ensemble de modeles)

Blending : Pondere par Sharpe ratio (sharpe_weighted) ou moyenne simple

Filtre : Exclusion des cibles dont le Sharpe ratio < 0.15

5. POST-PROCESSING

5.1 Ranking

Les predictions brutes sont converties en percentiles dans l'intervalle [0, 1] par era. Cette transformation assure une distribution uniforme des predictions, ce qui est requis par le format de soumission Numerai.

5.2 Neutralisation des features

NEUTRALISATION (neutralize_proportion = 0.5)

Methode : Projection least-squares pour supprimer la correlation des predictions avec les features individuelles.

Formule : $\text{predictions_neutralisees} = \text{predictions} - 0.5 * (\text{features} @ \beta)$

ou $\beta = (\text{features}^T @ \text{features})^{-1} @ \text{features}^T @ \text{predictions}$

Objectif : Reduire l'exposition aux features (factor exposure) pour ameliorer la generalisation et la stabilite des performances sur le long terme.

Intuition : Un modele trop correle aux features individuelles sera penalise par Numerai. La neutralisation force le modele a capturer des signaux plus subtils.

Flow du post-processing

```
Predictions brutes ----+
                        |
                        v
          +-----+
          | Least-Squares |
          | Projection    |
          | (proportion=0.5)|
          +-----+
          |
Features originales --->|
                        |
                        v
          Predictions neutralisees
                        |
                        v
          Ranking [0, 1]
                        |
                        v
          Submission.csv
```

6. PIPELINE DE PREDICTION (predict.py)

Etape	Action	Detail
1	Chargement modeles	Tous les modeles de base depuis models/
2	Chargement stacker	Ridge regression pre-entraine
3	Chargement donnees	numera_i_tournament_data.parquet (live)
4	Feature engineering	Memes settings que l'entrainement

5	Prediction par cible	Prediction -> Stacking -> Neutralisation -> Ranking
6	Export et upload	submission.csv + upload via NumerAPI

COHERENCE TRAIN / PREDICT

Le pipeline de prediction reprique exactement le feature engineering de l'entrainement (memes colonnes, memes transformations). Les colonnes sont validees via feature_columns.json et model_spec.json sauvegardes lors du training. Toute incoherence declencherait une erreur.

7. OPTIMISATION DES HYPERPARAMETRES

L'optimisation des hyperparametres est realisee avec Optuna, un framework d'optimisation bayesienne qui explore intelligemment l'espace des hyperparametres.

Parametre	Valeur
Framework	Optuna
Nombre de trials	50 (par default)
Fonction objectif	Sharpe ratio moyen par era sur holdout
Validation	CV temporelle (respecte l'ordre des eras)
Sortie	config/optimized_params.yaml
Strategie	TPE (Tree-structured Parzen Estimators)

8. TOURNOIS NUMERAI

Tournoi	Type	ID	Donnees	Soumission
Classic (t8)	Regression tabulaire	v5.2	~300k lignes, ~2000 feat.	prediction.csv
Signals (t11)	Ranking actions	v2.1	~7000 tickers, feat. custom	signals.csv
Crypto (t12)	Ranking crypto	v2.0	~100+ assets	submission

8.1 Concepts cles Numerai

Concept	Definition
Era	Periode temporelle (hebdomadaire Classic, quotidien Signals)
Target	Label de rendement cryptographiquement obscurci
Sharpe Ratio	Metrique principale : correlation par era, ajustee au risque
Correlation	Mesure entre predictions et target reel
Staking NMR	Allocation de tokens NMR pour recevoir des paiements
Neutralisation	Decorrelation des features pour stabilite du modele

9. NOEUDS DE PREDICTION CLOUD

Noeud	Tournoi	Modele	Fonctionnement
classic-node	Classic (t8)	salok1_classic.pkl	Download live data -> predict -> rank -> upload
signals-node	Signals (t11)	signals.pkl	Download signals -> predict par ticker -> upload
crypto-node	Crypto (t12)	salok1.pkl	Download crypto data -> predict -> upload
tgrv2-node	Classic (t8)	tgrv2.pkl	Variante TGRV2, meme flow que Classic

COMPOSITION DE CHAQUE NOEUD

Dockerfile : Image Python 3.11 legere optimisee pour la prediction

predict.py : Script de prediction specifique au tournoi

Modele .pkl : Modele pre-entraine (LightGBM/XGBoost + stacker)

requirements.txt : Dependances Python minimales

Declenchement : Webhook Numerai (quotidien apres test initial)

Flow d'execution d'un noeud de prediction

```
+-----+
|                                     |
|                               FLOW D'UN NOEUD                               |
|                                     |
+-----+
|                                     |
| Webhook Numerai (nouveau round disponible)                             |
|   |                               |                                       |
|   v                               |                                       |
| numerai-cli declenche le conteneur Docker                               |
|   |                               |                                       |
|   v                               |                                       |
| predict.py :                                                             |
|   |                               |                                       |
|   +---> 1. Download donnees live (NumerAPI)                             |
|   +---> 2. Chargement modele .pkl                                         |
|   +---> 3. Feature engineering                                             |
|   +---> 4. Prediction (ensemble + stacking)                             |
|   +---> 5. Post-processing (neutralisation + ranking)                   |
|   +---> 6. Upload predictions (NumerAPI)                                 |
|   |                               |                                       |
|   v                               |                                       |
| Conteneur se termine automatiquement                                     |
+-----+
```

10. AUTOMATISATION

10.1 Scripts PowerShell locaux

Script	Fonction
run_me.ps1	Pipeline complet : train + predict + upload
daily_full_setup.ps1	Config + train + deploy + test des 4 noeuds
daily_retrain_deploy.ps1	Retrain + deploy sans config/test
health_check.ps1	Test de sante des 4 noeuds
setup_plug_and_play.ps1	Installation one-command (setup complet)

10.2 Infrastructure as Code

L'ensemble de l'infrastructure AWS est provisionnee via Terraform (aws/main.tf). Le deploiement des conteneurs est gere par les scripts deploy.sh (Linux) et deploy.ps1 (Windows). Cette approche IaC garantit la reproductibilite et le versioning de l'infrastructure.

11. DIAGNOSTICS ET EVALUATION

Le module diagnostics.py fournit une evaluation complete des performances du modele apres l'entrainement.

Metrique	Description	Objectif
Correlation par era	Correlation predictions vs target par era	Maximiser
Sharpe Ratio	Correlation moyenne / ecart-type	Maximiser (> 0.15)
Max Drawdown	Pire perte cumulee consecutive	Minimiser
Exposition features	Correlation avec features individuelles	Minimiser (< 0.1)
Stabilite temporelle	Variance des performances inter-eras	Maximiser (stable)

12. STRUCTURE DU PROJET

Arborescence du projet numerIA

```
numerIA/
|
+--- numerai-project/
|   +--- src/                # 13 modules Python
|   |   +--- train.py        # Pipeline d'entrainement
|   |   +--- predict.py     # Pipeline de prediction
|   |   +--- models.py      # Definitions des modeles
|   |   +--- features.py    # Feature engineering
|   |   +--- stacking.py    # Logique de stacking
|   |   +--- neutralize.py  # Neutralisation des features
|   |   +--- diagnostics.py # Metriques et evaluation
|   |   +--- utils.py       # Utilitaires
|   |   +--- ...            # Autres modules
|   |
|   +--- config/            # Fichiers YAML de configuration
|   |   +--- training.yaml
|   |   +--- features.yaml
|   |   +--- program_input_params.yaml
|   |   +--- optimized_params.yaml
|   |
|   +--- classic-node/      # Noeud Classic (Dockerfile + predict.py + .pkl)
|   +--- signals-node/     # Noeud Signals
|   +--- crypto-node/      # Noeud Crypto
|   +--- tgrv2-node/       # Noeud TGRV2
|   |
|   +--- Dockerfile         # Image GPU principale
|   +--- entrypoint.sh      # Point d'entree AWS
|   +--- README.md
|
+--- aws/                  # Terraform
|   +--- main.tf            # Infrastructure principale
|   +--- variables.tf       # Variables
|   +--- outputs.tf        # Sorties
|
+--- run_me.ps1            # Pipeline local complet
+--- daily_full_setup.ps1  # Automatisation complete
+--- daily_retrain_deploy.ps1 # Retrain + deploy
+--- health_check.ps1     # Sante des noeuds
+--- setup_plug_and_play.ps1 # Installation one-command
```

13. QUESTIONS D'ENTRETIEN POSSIBLES ET REPONSES

CONSEILS POUR L'ENTRETIEN

Structurez vos reponses en 3 temps : **(1) Contexte/Probleme, (2) Solution/Implementation, (3) Resultats/Metriques**. N'hésitez pas a dessiner des schemas sur le tableau blanc pour illustrer l'architecture ou les flows de donnees.

Q1 : Pourquoi un ensemble multi-modeles plutot qu'un seul modele ?

Reponse : L'utilisation d'un ensemble reduit la variance des predictions et ameliore la robustesse. LightGBM (leaf-wise) et XGBoost (level-wise) ont des biais inductifs differents, ce qui apporte une diversite structurelle. Les 5 seeds LightGBM capturent la variabilite stochastique de l'entrainement. Le stacking Ridge apprend les poids optimaux de chaque modele plutot qu'une simple moyenne. Sur Numerai, la stabilite (Sharpe ratio) est plus importante que la correlation brute, ce qui favorise les ensembles.

Q2 : Comment evites-tu le data leakage temporel ?

Reponse : Trois mecanismes : (1) Cross-validation temporelle (time-series split) qui respecte l'ordre chronologique des eras. (2) Embargo de 4 eras entre train et validation pour eliminer l'autocorrelation a court terme. (3) Holdout de 50 eras recentes jamais vues pendant le tuning. Un split aleatoire classique serait catastrophique car les donnees financieres sont fortement auto-correlees temporellement.

Q3 : Pourquoi la neutralisation des features ?

Reponse : La neutralisation reduit l'exposition du modele aux features individuelles via une projection least-squares. Sans elle, le modele risque de sur-reagir a des features qui sont temporairement predictives mais instables. Numerai penalise les modeles a forte exposition aux features. Le parametre `neutralize_proportion=0.5` offre un compromis entre signal et stabilite. C'est une forme de regularisation post-hoc specifique au domaine financier.

Q4 : Comment geres-tu les couts cloud ?

Reponse : Plusieurs strategies : (1) EC2 Spot instances (~70% moins cher que on-demand). (2) Auto-termination apres execution. (3) Modeles pre-entraines localement (GPU RTX 4060) pour limiter l'usage cloud. (4) Images Docker legeres (Python 3.11 minimal). (5) S3 avec retention 90 jours. (6) Lambda + EventBridge pratiquement gratuits. Cout total : 0.40-0.80 EUR par execution hebdomadaire.

Q5 : Quelle est ta strategie de validation ?

Reponse : Validation a 3 niveaux : (1) CV temporelle 5-fold avec embargo pendant l'entrainement. (2) Holdout de 50 eras recentes pour l'evaluation finale et le tuning Optuna. (3) Metriques de diagnostics post-entrainement (Sharpe, drawdown, exposition). Le Sharpe ratio par era est la metrique principale car il capture la performance ajustee au risque, plus pertinente que la correlation brute.

Q6 : Comment assures-tu la reproductibilite ?

Reponse : (1) Seeds fixes pour tous les modeles (42-46). (2) Configurations YAML versionnees. (3) Metadata sauvegardees (model_spec.json, feature_columns.json). (4) Docker pour l'isolation de l'environnement. (5) Terraform pour l'infrastructure. (6) Artefacts S3 avec retention. Chaque run produit un ensemble de fichiers qui permet de reproduire exactement les memes resultats.

Q7 : Pourquoi LightGBM comme modele principal ?

Reponse : LightGBM offre le meilleur compromis performance/vitesse pour les donnees tabulaires denses comme celles de Numerai (~2000 features, ~300k lignes). La croissance leaf-wise est plus efficace que level-wise pour ce type de donnees. Le support GPU natif (CUDA) accelere l'entrainement. feature_fraction=0.1 force la diversite entre les arbres. Les benchmarks sur Numerai montrent que LightGBM domine les solutions gagnantes.

Q8 : Comment geres-tu le deploiement automatique ?

Reponse : Le deploiement suit une chaine automatisee : (1) Entrainement local avec sauvegarde .pkl. (2) Build Docker avec le modele embarque. (3) Push vers ECR. (4) numerai-cli configure les webhooks Numerai. (5) A chaque nouveau round, le webhook declenche le conteneur. (6) Pour l'entrainement cloud, EventBridge declenche Lambda qui demarre EC2 Spot avec auto-termination. Scripts PowerShell pour orchestrer le tout.

Q9 : Quelle est la difference entre les tournois Classic, Signals et Crypto ?

Reponse : **Classic (t8)** : regression tabulaire pure sur ~2000 features obscurcies, ~300k lignes, resolution hebdomadaire. **Signals (t11)** : ranking d'actions reelles (~7000 tickers), features custom possibles, resolution quotidienne. **Crypto (t12)** : ranking de crypto-actifs (~100+ assets), donnees specifiques crypto. Classic est le plus competitif et le mieux documente. Signals permet d'apporter ses propres features. Crypto est le plus recent et le plus volatil.

Q10 : Comment optimises-tu les hyperparametres ?

Reponse : Optuna avec 50 trials par default, strategie TPE (Tree-structured Parzen Estimators). L'objectif est le Sharpe ratio moyen par era sur le holdout, pas la correlation brute, car on cherche la stabilite. La CV temporelle est respectee pendant l'optimisation. Les meilleurs parametres sont sauvegardes dans optimized_params.yaml et appliques automatiquement au prochain entrainement.

RECAPITULATIF RAPIDE

Pipeline end-to-end numerIA

```
+-----+
|                FLOW COMPLET numerIA                |
+-----+
|
| [DONNEES]  Numerai API                               |
|   |        ~300k lignes, ~2000 features (parquet)   |
|   v                                               |
| [FEATURES] Selection + Engineering (+37 features)   |
|   |                                               |
|   v                                               |
| [VALIDATION] CV temporelle 5-fold + embargo 4 eras   |
|   |                                               |
|   v                                               |
| [ENTRAINEMENT] 5x LightGBM + 1x XGBoost (+ Ridge, MLP opt.) |
|   |                                               |
|   v                                               |
| [STACKING] Ridge regression sur predictions OOF      |
|   |                                               |
|   v                                               |
| [POST-PROCESS] Neutralisation (0.5) + Ranking [0,1]  |
|   |                                               |
|   v                                               |
| [DEPLOIEMENT] Docker -> ECR -> 4 noeuds cloud       |
|   |                                               |
|   v                                               |
| [SOUMISSION] NumerAPI -> Classic + Signals + Crypto  |
|   |                                               |
|   v                                               |
| [MONITORING] Diagnostics + CloudWatch + Health checks |
+-----+
```

Chiffres clés a retenir

Metrique	Valeur
Features originales	~2000
Features creees	37
Lignes d'entrainement	~300k
Modeles de base	6 (5 LightGBM + 1 XGBoost)
Folds CV	5 + embargo 4 eras
Holdout	50 eras recentes
Cibles max	20
Noeuds cloud	4
Cout hebdomadaire	0.40-0.80 EUR
Neutralisation	proportion = 0.5

Early stopping	200 rounds
Optuna trials	50
Modules Python	13

BONNE CHANCE POUR TON ENTRETIEN !

Ce document couvre l'ensemble du projet numerIA. Revois les diagrammes d'architecture, les choix de modeles et leurs justifications, ainsi que les strategies de validation. N'oublie pas de mentionner les aspects DevOps/MLOps (Docker, Terraform, CI/CD) qui demontrent une approche professionnelle du deploiement ML.